PROYECTO DE APLICACIÓN

Nexo

Gestión visual de comunidades conectada a Supabase

MEMORIA TÉCNICA Y FUNCIONAL
Aplicación web y Android

Ionic · Angular · Capacitor · Supabase

Resumen ejecutivo

Nexo permite administrar comunidades y ofrecer a cada usuario un portal privado con la información de su propia comunidad. Incluye autenticación, permisos por rol, fichas visuales, fotografías privadas en Supabase Storage, actualizaciones en tiempo real y exportación de informes. El cliente web se empaqueta para Android mediante Capacitor.

Objetivos

- Reunir los datos operativos y económicos en una interfaz clara y adaptable a móvil.
- Separar el acceso de administración del acceso de cada comunidad, aplicando autorización también en PostgreSQL mediante Row Level Security (RLS).
- Gestionar fotos de portal y exportar informes desde navegador o mediante la hoja nativa de compartir de Android.

Alcance de esta memoria

Describe el código del repositorio, el modelo de acceso, los pasos para configurar Supabase y Android, y las comprobaciones realizadas. La compilación nativa está verificada; el backend remoto y un teléfono físico requieren validación con las cuentas y dispositivos del propietario.

Fecha de generación: 31/10/2026

1. Arquitectura y datos

Ionic y Angular construyen una aplicación de una sola página; Capacitor incorpora la interfaz en Android. Supabase proporciona autenticación, PostgreSQL, almacenamiento de imágenes y funciones de servidor.

Capa	Responsabilidad
Interfaz	Páginas Angular para login, resumen de administración y comunidades; componentes Ionic adaptados a pantalla táctil.
Servicios cliente	SupabaseService gestiona sesión y perfil; CommunitiesService consulta y modifica registros, carga imágenes firmadas y procesa eventos Realtime.
Backend	Supabase Auth identifica usuarios; PostgreSQL/RLS autoriza filas; Storage guarda fotos en un bucket privado; una Edge Function crea/vincula accesos.
Android	Capacitor 8 empaqueta www y enlaza plugins nativos, incluyendo Filesystem y Share para los informes.

Entidades principales

Entidad	Campos relevantes	Uso
comunidad	id, comunidad, numero, email, coste_mensual, junta, cancelada, user_id, portal_image_path	Datos de cada comunidad y vínculo del usuario. La imagen se guarda en Storage; la tabla almacena su ruta.
profiles	id, full_name, role, community_id	Perfil relacionado con auth.users. role distingue admin/user; community_id vincula cuenta y comunidad.
Storage: community-photos	Objetos bajo la carpeta del ID de comunidad	Bucket privado; JPEG/PNG/WebP hasta 5 MB; lectura autorizada mediante URL firmada.

Flujo de acceso

- El usuario inicia sesión con Supabase Auth y se carga su perfil.
- RLS limita la consulta: administradores pueden consultar todas las comunidades; usuarios estándar solo la comunidad vinculada.
- La interfaz representa la información en una ficha; las fotos se leen mediante URL firmadas de duración limitada.
- Las modificaciones de datos e imágenes también se autorizan en Supabase, no solo en la aplicación.

2. Seguridad y experiencia de uso

Autorización y privacidad

Las migraciones habilitan RLS en profiles y comunidad. La función is_admin() comprueba el rol y las políticas diferencian lectura y escritura: los administradores gestionan registros, mientras que las cuentas normales leen su comunidad. La migración de usuarios añade user_id a comunidad y community_id a profiles. Las políticas del bucket privado aplican separación equivalente a las fotografías.

La clave sb_publishable configurada para el cliente no sustituye las políticas RLS. La clave service_role solo debe existir como secreto de la Edge Function y nunca incluirse en Angular, en la APK ni en el repositorio.

Creación y asociación de cuentas

La función create-community-user valida que la sesión invocante pertenezca a un administrador, invita por correo cuando es necesario, asigna un perfil estándar y enlaza usuario y comunidad. La entrega del correo depende de la configuración de Auth/SMTP y de que la función esté desplegada con sus secretos de servidor.

Interfaz y capacidades

- Resumen administrativo con indicadores, gráfico y exportación CSV/PDF.
- Directorio responsive con búsqueda y filtros, edición de comunidades, estado y fotografía del portal.
- Portal individual con presentación por atributos, fotografía y nota de privacidad, no como tabla administrativa.
- Modo oscuro disponible en el inicio de sesión y en la navegación; la preferencia se conserva localmente.
- Mensajes visibles de error y estados de sincronización para operaciones de carga y actualización.

Actualización y exportación

Realtime refleja cambios de comunidad en clientes conectados. Los informes CSV y PDF se generan localmente. En navegador se descargan; en Android se escriben en el directorio temporal privado de la app y se pasan al diálogo nativo de compartir o guardar.

Protección de fotos

Las fotos no son públicas. El bucket community-photos limita tamaño y formatos; la subida, sustitución y eliminación se reserva a administradores. El cliente solicita enlaces firmados conforme a las políticas de Storage.

3. Aplicación Android

La app reutiliza el frontend Ionic Angular con Capacitor. El identificador de paquete propio es **es.proyecto.nexocomunidades**, distinto del usado por la otra app TablaDatos. La configuración requiere Android SDK API 36 para compilar; el mínimo configurado es API 24. Auth, datos y fotos requieren Internet.

Entorno necesario

- Node.js y npm para instalar dependencias y compilar Angular.
- Android Studio, Android SDK (API 36 y herramientas correspondientes) y JDK 21.
- Configurar ANDROID_HOME y ANDROID_SDK_ROOT con la ruta del SDK; añadir platform-tools al PATH para utilizar adb.
- Para ejecutar en un teléfono: activar opciones de desarrollador y depuración USB y autorizar el ordenador. Para publicar: firmar desde Android Studio con una clave protegida fuera del repositorio.

Preparación de Supabase

- Aplicar en orden las migraciones de supabase/migrations: esquema y roles, asociación de usuarios y fotos. Comprobar que no queden políticas antiguas de acceso anónimo o lectura general.
- Crear la cuenta administrativa y asignarle role = admin en profiles. Crear usuarios de comunidad o habilitar invitaciones.
- Desplegar supabase/functions/create-community-user y verificar secretos. Configurar el correo de Auth y las URL permitidas para invitaciones.
- Comprobar tabla/columnas, RLS en comunidad y profiles, bucket privado community-photos y políticas de Storage.
- Verificar que src/environments/environment.ts y environment.prod.ts apuntan al proyecto Supabase esperado; no incluir claves secretas.

Comandos

npm ci - instala dependencias bloqueadas por package-lock.json.
npm start - inicia Angular en navegador.
npm run android:sync - compila la interfaz web y sincroniza archivos/plugins con android/.
npm run android:open - abre el proyecto en Android Studio.
npm run lint y **npm test -- --watch=false** - verifican código y pruebas.

Si Android Studio no encuentra el SDK, configura las variables de entorno y vuelve a sincronizar. La APK de depuración suele quedar en android/app/build/outputs/apk/debug/app-debug.apk.

4. Verificación y entrega

Validaciones realizadas

Comprobación	Resultado
npm run build	Correcto. Angular genera la app en www. Hay advertencias no bloqueantes de dependencias CommonJS de canvg/jspdf.
npm test -- --watch=false	Correcto: 3 archivos de pruebas, 4 pruebas satisfactorias.
npm run lint	Correcto: todos los archivos analizados pasan lint.
npm run android:sync	Correcto: assets web y seis plugins Capacitor sincronizados con android/.
Gradle assembleDebug	Correcto: BUILD SUCCESSFUL; APK de depuración compilada con el SDK local.

Pasos para la puesta en marcha

- Instalar Node.js, JDK y Android Studio/SDK; fijar ANDROID_HOME y ANDROID_SDK_ROOT de forma persistente.
- Preparar Supabase aplicando las migraciones, validar RLS y publicar la Edge Function con secretos.
- Probar una cuenta administradora y una estándar vinculada; comprobar que la cuenta estándar no puede leer otras comunidades.
- Ejecutar en navegador y Android; probar modo oscuro, fotos, conectividad, formularios y exportación/compartición.
- Para distribución, configurar firma de release, versión y canal de publicación.

Límites de la validación

La compilación confirma que el proyecto nativo, los plugins y el código se integran. No acredita una prueba manual en dispositivo, el envío real de invitaciones ni la disponibilidad de tablas, políticas, Storage y función en el backend remoto. Esos pasos requieren acceso a Supabase y credenciales del propietario.

Archivos clave

Área	Ruta
Configuración Android	capacitor.config.ts; android/; package.json
Conexión y datos	src/app/services/supabase.service.ts; src/app/services/communities.service.ts
Informes nativos/web	src/app/services/report.service.ts
SQL	supabase/migrations/
Invitaciones	supabase/functions/create-community-user/index.ts
Guía de desarrollo	README.md

Documento generado a partir del código y configuración del repositorio TablaDatosSupabase. Mantener esta memoria y el README alineados con cambios de esquema, permisos o versión móvil.